Embedded Data Structure
(Lecture Note 16)

Sorting Algorithms (3)

2025. 5. 6

Prof. Wonjun Kim

School of Electrical and Electronics Engineering

Review of the Last Class (1/2)

- **We learned about simple sorting algorithms**
  - **Four types of sorting algorithms as follows :**

Selection sort :

- Change with minimum
- Performance : normal
- Complexity : $O(N^2)$

Bubble sort :

- Find min. with sorting
- Performance : bad
- Complexity : $O(N^2)$

Insertion sort :

- Sorting until current pos.
- Performance : good
- Complexity : $O(N^2)$

Shell sort :

- Sorting with interval
- Performance : good
- Complexity : $O(N^{1.25})$

Review of the Last Class (2/2)

KU KONKUK UNIVERSITY

▪ Generalization of sorting algorithms

- **We use the void pointer for all the type of data**

Sort only for "int"	Sort for all the types
Store key : int min	void *min; min = malloc(width);
Array elem : a[y]	*(type)((char *)base + y*width)
Comparison : min > a[x]	fcmp(min, (char *)base + x*width) > 0
Assign : a[y] = min	memcpy((char *)base + y*width, min, width)

※ What does "width" mean in the table ?

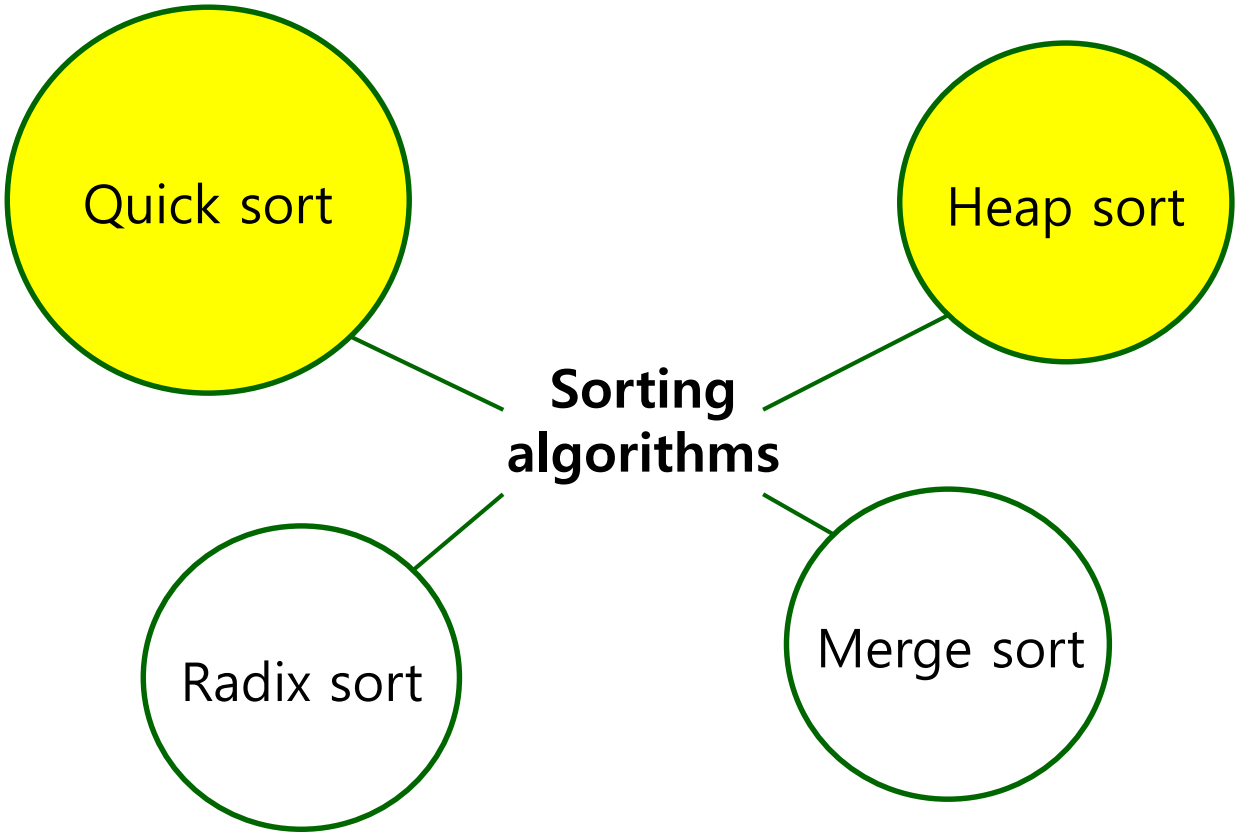
* Why do we need to conduct the memory allocation ?

* Remember the method for "**pointer casting**"

➡ **Try to implement the insertion sort and other ones !**

Overview of Today Class

- **Advanced algorithms for sorting**
  - **Key operation : divide and conquer**



※ Some of them require the additional memory space

Quick Sort (1/7)

- **Overall procedure of quick sort**
  - **Divide and sort (carefully follow the procedure !)**

i → T O L E A R N S O R T A L G O R I T H **M** ← j

Pseudo code :

1) If n > 1, (# of elements)
    1) Partition (divide) array a[] whose size is N, and
       Exchange the position of pivot to "mid"
    2) Quick sort(a, mid);
    3) Quick sort(a+mid+1, N-mid-1);

- There are two index, i.e., i and j for moving left→right and right→left

- If i >= j, then exchange the pos. of pivot to i

  (see the example in the following slide !)

Quick Sort (2/7)

- **Example of quick sort**
  - **Divide and sort (carefully follow the procedure !)**

Pivot

T O L E A R N S O R T A L G O R I T H **M**

H O L E A R N S O R T A L G O R I T T **M**

H I L E A R N S O R T A L G O R O T T **M**

H I L E A G N S O R T A L R O R O T T **M**

H I L E A G L S O R T A N R O R O T T **M**

H I L E A G L A O R T S N R O R O T T **M**

H I L E A G L A **M** R T S N R O R O T T O

Smaller than "M" **Larger than "M"**

Quick Sort (3/7)

- **Example of quick sort – cont'd**
  - **Divide and sort (carefully follow the procedure !)**

H I L E A G L **A**

A I L E H G L **A**

A L E H G L **I**

G E H L L **I**

G E **H** I L

G **E** H

E G

M R T^2 S N R O R O T^3 T^1 **O**

M O T^2 S N R O R R T^3 T^1 **O**

M O O S N R T^2 R R T^3 T^1 **O**

M O O N S R T^2 R R T^3 T^1 **O**

M O O **N** O S

N O O

O R T^2 R R T^3 T^1 **S**

O R R R T^2 T^3 T^1 **S**

O R R **R** S T^2

O R R T^3 T^1 **T^2**

T^1 T^3 **T^2**

Quick Sort (4/7)

- **Implementation**
  - **Based on pseudo code with recursion**

```
void quick_sort(int *a, int N)
{
    int v, t;
    int i, j;
    if(N > 1) // termination
    {
        v = a[N-1];  ⟶ Pivot
        i = -1;
        j = N-1;
        while(1){
            while(a[++i] < v);
            while(a[--j] > v);
            if(i >= j) break;
            t = a[i];
            a[i] = a[j];
            a[j] = t;
        }
        t = a[i];
        a[i] = a[N-1];
        a[N-1] = t;

        quick_sort(a, [      ]);
        quick_sort(a+i+1, [      ]);
    }
}
```

Think about these statements

What does it mean ?

Quick Sort (5/7)

KU KONKUK UNIVERSITY

- **Characteristics of quick sort**
  - **Relation with the depth of recursion (when is the best ?)**
  - **It requires the internal stack due to the call of recursion**

    ➔ **How about big-O notation ? NlogN**

※ Analysis of the processing time (using the tick)

N	Sorted	Random	Half-sorted	Reverse
10	0	0	0	0
100	0	0	0	1
500	8	1	2	10
1000	34	2	3	40
2000	135	5	5	162

Why is it slow using the sorted data ?

Quick Sort (6/7)

- **Non-recursive version of quick sort**
  - **Type 1 conversion (see recursion class)**

```
void quick_sort1(int *a, int N)
{
    int v, t;
    int i, j;
    int l, r;
    init_stack();
    l = 0;
    r = N-1;
    push(r);
    push(l);
    while(!is_stack_empty()){
        l = pop();
        r = pop();
        if(r-l+1 > 1) // termination
        {
            v = a[r];
            i = l-1;
            j = r;
            while(1){
                while(a[++i] < v);
                while(a[--j] > v);
                if(i >= j) break;
                t = a[i];
                a[i] = a[j];
                a[j] = t;
            }
            t = a[i];
            a[i] = a[r];
            a[r] = t;
        }
    }
}
```

Why do we use it ?

Push (four statements)

Quick Sort (7/7)

Implementation

- **Quick sort with original and non-recursive versions**

```
#include <stidio.h>
#include <stdlib.h>

void main()
{
   int a[15] = {9, 4, -80, 1, 172, 13, 9, 39, -99, 2054, 10, -10, 3, 67, -28};

   quick_sort(a, 15);      // recursive version
   quick_sort1(a, 15);    // non-recursive version

   // 1. compare the processing time between two functions
   // 2. generate random >2000 variables and compare with other methods
}
```

Improve Quick Sort ! (1/3)

- **Partition with random variables**
  - **Right-most pivot → randomly selected pivot**

※ Think about the sorted case with the right-most pivot !

```
...
while(!is_stack_empty()){
        l = pop();
        r = pop();
        if(r-l+1 > 1) // termination
        {
            t = rand()%N;
            v = a[t];
            a[t] = a[r];
            a[r] = v;
            i = l-1;
            j = r;
            while(1){
...
```

Simply modify the pivot using the random value from [0, N-1]

Check the processing time !

Improve Quick Sort ! (2/3)

- **Quick sort with insertion sort**
  - **Large interval → quick sort / Small → interval → insertion sort**

```
void quick_sort3(int *a, int N)
{
    int v, t;
    int i, j;
    int l, r;
    init_stack();
    l = 0;
    r = N-1;
    push(r);
    push(l);
    while(!is_stack_empty()){
        l = pop();
        r = pop();
        if(r-l+1 > 200) // quick sort
        {
            t = rand()%N;
            v = a[t];
            a[t] = a[r];
            a[r] = v;
            i = l-1;
            j = r;
            while(1){
                while(a[++i] < v);
                while(a[j--] > v);
                if(l >= j) break;
                t = a[i];
                a[i] = a[j];
                a[j] = t;
            }
            t = a[i];
            a[i] = a[r];
            a[r] = t;
            [                ]
        }
        else
            insert_sort( [                ] );
    }
}
```

Push (four statements)

Improve Quick Sort ! (3/3)

- **Improvement of processing time**

Random pivot

N	Sorted	Random	Half-sorted	Reverse
10	0	0	0	1
100	1	1	0	0
500	2	2	2	2
1000	4	4	4	4
2000	9	9	8	8

With insertion

N	Sorted	Random	Half-sorted	Reverse
100	0	1	0	1
500	1	3	2	1
1000	1	6	4	2
2000	2	11	12	3
5000	5	33	29	6
7000	8	43	44	10
10000	11	61	62	14

Summary

- **Sorting algorithms : data management**
  - **There is no optimal solution for all the cases**
  - **Remember the generalization scheme using void pointers**
  - **Today's topic : insertion / shell / distribution counting**

- **In the next class, we will learn more about quick sort**